PATTERNS & INTUITIONS

Mastering Data Structures & Algorithms

Stop memorizing problems. Learn the ~20 recurring patterns, the intuition behind each, and how to recognize which one a problem is secretly asking for.

Foundations · pattern recognition · templates · complexity · pitfalls
A practitioner's reference for interviews and for thinking like an algorithm designer

Contents

1 · How to use this guide

There are thousands of DSA problems but only about twenty underlying *patterns*. Experts don't recall a solution for every problem — they recognize which pattern a problem belongs to, then apply a template they understand deeply. This guide teaches that recognition skill.

For each pattern you get four things: the **intuition** (why it works, as a mental model), the **triggers** (the words and structures in a problem that signal it), a **template** you can adapt, and the **complexity and pitfalls**. Read it actively: for every pattern, try to recall a problem you've seen that fits it.

INTUITION
The mental model — the one sentence that makes the pattern obvious.

WHEN TO USE IT
The recognition signals — phrases and structures that should make you think of this pattern.

COMPLEXITY
The time/space cost and why.

PITFALL
The classic mistake that turns a correct idea into a wrong or slow solution.

THE META-SKILL
The fastest way to improve is to **name the pattern before you code**. If you can't name it, you don't yet understand the problem — re-read it looking for the triggers in §4. Coding before naming is the #1 reason people get stuck.

2 · Complexity foundations

Big-O describes how runtime or memory *grows* as input size `n` grows — it's about the shape of the curve, not the exact count. Mastering patterns means instinctively knowing the cost of what you're writing.

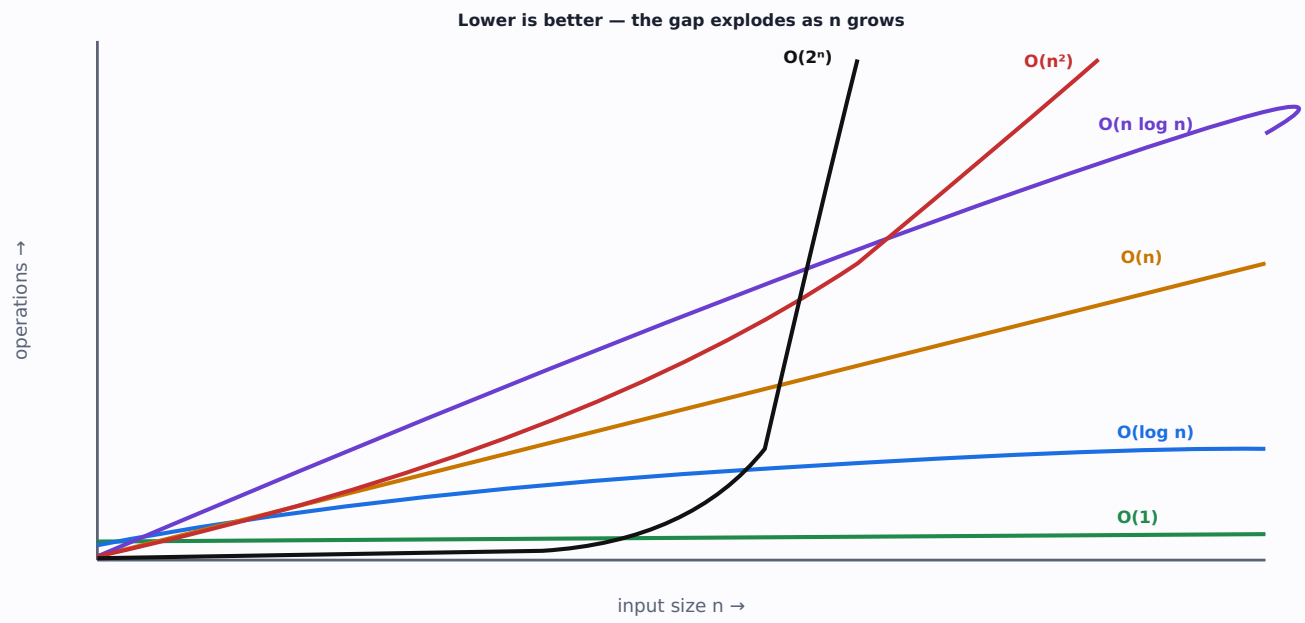


Figure 1 — The growth hierarchy. The practical lesson: turning an O(n²) approach into O(n log n) or O(n) is usually the entire point of a pattern. Exponential O(2ⁿ) is fine only for small n (subsets, backtracking).

Class	Name	Typical source
O(1)	Constant	Hash lookup, array index, stack push
O(log n)	Logarithmic	Binary search, balanced-tree / heap operation
O(n)	Linear	One pass; two pointers; sliding window
O(n log n)	Linearithmic	Efficient sorting; heap of n items; divide & conquer
O(n²)	Quadratic	Nested loops over the input (often the brute force to beat)
O(2ⁿ), O(n!)	Exponential / factorial	All subsets / all permutations (backtracking)

TWO IDEAS PEOPLE FORGET

Amortized cost: a dynamic array `append` is O(1) *amortized* — occasional resizes average out. **Space includes the call stack:** a recursion `n` deep costs O(n) space even if it allocates nothing, which matters for deep trees and DP.

3 · Core data structures & their costs

Patterns are built on data structures. Choosing the right one is often 80% of the solution — the pattern just exploits its strengths.

Structure	Access	Search	Insert	Delete	Superpower
Array / dynamic array	O(1)	O(n)	O(n)*	O(n)	Indexing, cache locality
Hash map / set	—	O(1) avg	O(1) avg	O(1) avg	Instant lookup by key
Linked list	O(n)	O(n)	O(1)†	O(1)†	Splice without shifting
Stack / queue	—	—	O(1)	O(1)	LIFO / FIFO ordering
Heap (priority queue)	O(1) peek	—	O(log n)	O(log n)	Always know the min/max
Balanced BST / sorted set	—	O(log n)	O(log n)	O(log n)	Ordered + range queries
Trie	—	O(L)	O(L)	O(L)	Prefix sharing (L = key length)
Union-Find (DSU)	—	~O(α)	~O(α)	—	Near-constant connectivity
Graph (adjacency list)	—	—	—	—	Model relationships

* O(1) amortized appending at the end. † Once you already hold the node; finding it is O(n). α = inverse-Ackermann, effectively a small constant.

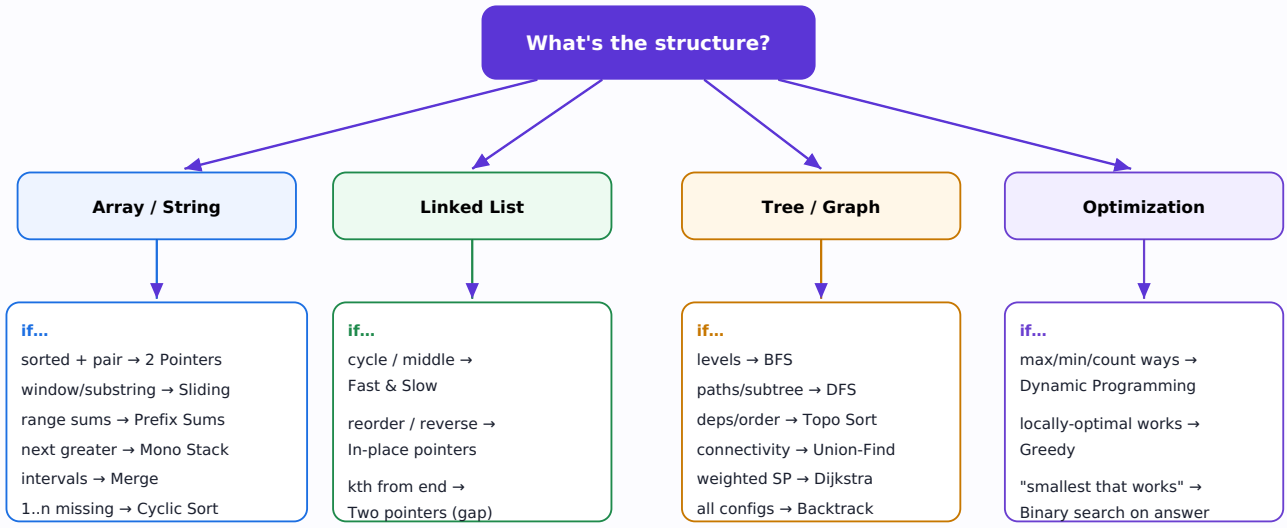
INTUITION — THE HASH MAP IS YOUR DEFAULT TOOL

A huge fraction of "make it faster" reduces to: *store something in a hash map so a future lookup becomes O(1) instead of O(n).* Pair-sum, seen-before, frequency counts, grouping, caching — all the same move. When you see a nested loop searching for a value, ask "could a hash map remember this?"

4 · How to recognize the pattern

This is the core mastery skill: mapping the *surface features* of a problem to the pattern underneath. Build the reflex with this map, then the decision tree.

The problem mentions…	Reach first for…
Sorted array + find a pair / triplet / target sum	Two Pointers (or Binary Search)
Contiguous subarray / substring with a constraint (longest, shortest, ≤ k)	Sliding Window
Linked list cycle, or find the middle	Fast & Slow Pointers
"k largest / smallest / most frequent"	Heap (Top-K)
Running median of a stream	Two Heaps
"Generate all subsets / permutations / combinations", constraint puzzles	Backtracking
"Max / min / number of ways" with sequential choices	Dynamic Programming
Intervals / overlapping ranges / meeting rooms	Merge Intervals (sort + sweep)
Dependencies, prerequisites, build order (a DAG)	Topological Sort
Connectivity, number of components, cycle in undirected graph	Union-Find or DFS
Shortest path, *unweighted*	BFS
Shortest path, *weighted* (non-negative)	Dijkstra
"Next greater / previous smaller element", histogram	Monotonic Stack
Many range-sum queries	Prefix Sums
Array holds numbers 1..n, find missing / duplicate	Cyclic Sort
"Find the smallest/largest value that works", or O(log n) demanded	Binary Search on the answer
Prefixes / autocomplete / word dictionary	Trie
"Single number", subset enumeration, no extra space	Bit Manipulation



Then confirm with the trigger words above — most problems reveal their pattern in one or two phrases.

Figure 2 — A pattern picker. Start from the data you're given and the goal, narrow to a family, then match trigger words. With practice this collapses into instant recognition.

5 · Two Pointers array / string

INTUITION

Two indices replace a nested loop. By moving a *left* and *right* pointer intelligently — toward each other on a sorted array, or one chasing the other — you explore in one pass what brute force does in $O(n^2)$.

WHEN TO USE IT

A **sorted** array and you need a pair/triplet summing to a target; palindrome checks; reversing in place; removing duplicates in place; merging two sorted sequences; partitioning.

```
# Pair summing to target in a SORTED array
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == target: return (l, r)
    elif s < target: l += 1      # need bigger → move left up
    else:            r -= 1      # need smaller → move right down
```

COMPLEXITY

O(n) time, O(1) space — after an O(n log n) sort if the input wasn't already sorted.

PITFALL

The opposite-ends variant needs sorted data; on unsorted input use a hash map instead. For triplets, fix one element and two-point the rest, and skip duplicates to avoid repeated answers.

6 · Sliding Window array / string

INTUITION

Instead of recomputing a value for every subarray, keep a moving window and *update incrementally*: add the element entering on the right, remove the one leaving on the left. The work each step is O(1), so the whole scan is O(n).

WHEN TO USE IT

Anything about a **contiguous** subarray/substring — longest/shortest/maximum/minimum subject to a constraint (sum ≤ k, at most k distinct chars, no repeats). Fixed-size windows when k is given; variable-size when you optimize a length.

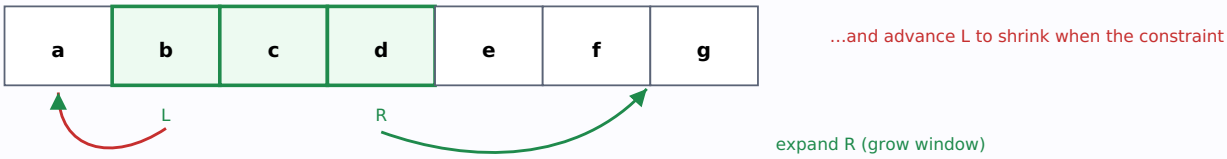


Figure 3 — The window [L, R] slides right; when it violates the constraint, L advances to restore it. Each index enters and leaves at most once → O(n).

```
# Longest substring with at most K distinct characters
from collections import Counter
cnt, l, best = Counter(), 0, 0
for r, ch in enumerate(s):
    cnt[ch] += 1
    while len(cnt) > K:           # window invalid → shrink from left
        cnt[s[l]] -= 1
        if cnt[s[l]] == 0: del cnt[s[l]]
        l += 1
    best = max(best, r - l + 1)   # window valid → record answer
return best
```

COMPLEXITY

O(n) time, O(k) space. Each element is added once and removed once.

PITFALL

Be precise about *when* you record the answer (inside vs after shrinking) and whether the window must always be valid (shrink eagerly) or you want the max valid window. Mixing these gives off-by-one bugs.

7 · Fast & Slow Pointers linked list

INTUITION

Two pointers moving at different speeds (one +1, one +2). On a loop, the fast one laps the slow one and they *must* meet — like runners on a circular track. On a line, the fast one reaching the end leaves the slow one exactly at the middle.

WHEN TO USE IT

Detect a cycle in a linked list; find the cycle's start; find the middle node; "happy number"; find a duplicate in an array treated as a linked list (values as next-pointers).

```
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:        # they met → cycle exists
        return True
return False                # fast hit the end → no cycle
```

COMPLEXITY

O(n) time, O(1) space — the key win over a hash set of visited nodes.

PITFALL

Guard `fast and fast.next` before the double hop or you'll dereference null. To find the cycle's *start*, reset one pointer to head after they meet and advance both by one.

8 · Binary Search searching

INTUITION

If you can ask a yes/no question whose answer is **monotonic** across the search space (false... false... true... true), you can binary-search the boundary, halving the space each step. This applies to sorted arrays *and* to "the smallest value that works" — searching over the answer itself.

WHEN TO USE IT

Sorted input; "find first/last position"; an O(log n) requirement; or "minimize the maximum / maximize the minimum / smallest feasible X" where feasibility is monotonic (binary search on the answer).

```
# Generic "leftmost value where predicate(x) becomes True"
lo, hi = min_x, max_x
while lo < hi:
    mid = (lo + hi) // 2
    if predicate(mid): hi = mid      # mid works → answer is mid or lower
    else:              lo = mid + 1  # mid fails → go higher
return lo
```

COMPLEXITY

O(log n) for array search; O(n·log(range)) when the predicate itself is an O(n) feasibility check over the value range.

PITFALL

Boundary bugs dominate: pick a consistent invariant (e.g. "answer is in [lo, hi]"), decide `<` vs `<=` and whether mid maps to `hi=mid` or `lo=mid+1`, and make sure the space strictly shrinks each iteration to avoid infinite loops.

9 · Prefix Sums array

INTUITION

Precompute cumulative totals once; then any range sum is a single subtraction `P[r+1] − P[l]`. You trade O(n) preprocessing for O(1) queries. Combine with a hash map of previously-seen prefix sums to count subarrays with a target sum.

WHEN TO USE IT

Repeated range-sum queries; "subarray sums to k"; running balances; 2D region sums (integral image); difference arrays for range updates.

```
# Count subarrays summing to k
from collections import defaultdict
seen = defaultdict(int); seen[0] = 1
running = ans = 0
for x in a:
    running += x
    ans += seen[running - k]   # a prior prefix makes a window summing to k
    seen[running] += 1
return ans
```

COMPLEXITY

O(n) build and O(1) per range query; the hash-map variant is O(n) overall.

PITFALL

Off-by-one with the leading zero: define `P[0]=0` so `P[i]` is the sum of the first `i` elements. Seed `seen[0]=1` so a prefix that itself equals k is counted.

10 · Merge Intervals array

INTUITION

Sort intervals by start, then sweep left to right: if the next interval overlaps the current one, extend; otherwise close the current and start fresh. Sorting turns a tangle of overlaps into a single clean pass.

WHEN TO USE IT

Merging/inserting intervals; meeting-room counts; free-time gaps; any overlapping-ranges question.

```
intervals.sort(key=lambda x: x[0])
merged = [intervals[0]]
for s, e in intervals[1:]:
    if s <= merged[-1][1]:                # overlap
        merged[-1][1] = max(merged[-1][1], e)
    else:
        merged.append([s, e])
```

COMPLEXITY

O(n log n), dominated by the sort; O(n) extra space.

PITFALL

Decide whether touching endpoints (`[1,2]` and `[2,3]`) count as overlapping for your problem. For "minimum rooms", a min-heap of end times (or a start/end sweep line) is cleaner than merging.

11 · Cyclic Sort array

INTUITION

When an array is a permutation of `1..n` (or `0..n−1`), each value has a known home index. Swap each element to its home in one pass; afterwards any index whose value is wrong reveals the missing/duplicate number — all in O(n) and O(1) extra space.

WHEN TO USE IT

"Array of n numbers in range [1,n]"; find the missing number, the duplicate, all missing, or the first missing positive.

```
i = 0
while i < n:
    home = a[i] - 1                      # where a[i] belongs
    if 0 <= home < n and a[i] != a[home]:
        a[i], a[home] = a[home], a[i]    # place it; don't advance
    else:
        i += 1
# now any i with a[i] != i+1 is anomalous
```

COMPLEXITY

O(n) time (each value reaches home at most once), O(1) extra space.

PITFALL

Only advance `i` when the current slot is already correct, else you'll skip elements. Watch the index base (1..n vs 0..n−1) and guard out-of-range values for "first missing positive".

12 · Monotonic Stack `stack`

INTUITION

Keep a stack whose values stay sorted (increasing or decreasing). When a new element breaks the order, pop — and each pop *answers* a "next greater/smaller" relationship. Every element is pushed and popped once, so an apparent O(n²) scan becomes O(n).

WHEN TO USE IT

"Next/previous greater or smaller element"; daily temperatures; stock span; largest rectangle in a histogram; trapping rain water.

```
# Next greater element to the right (indices)
res = [-1] * n
stack = []                      # holds indices, values decreasing
for i, x in enumerate(a):
    while stack and a[stack[-1]] < x:
        res[stack.pop()] = i    # x is the next greater for that index
    stack.append(i)
```

COMPLEXITY

O(n) time, O(n) space — amortized because each index is pushed/popped at most once.

PITFALL

Choose increasing vs decreasing based on whether you want next-greater or next-smaller, and store indices (not values) when you need distances. For circular arrays, iterate `2n` with modulo.

13 · Heaps & Top-K heap

INTUITION

A heap gives you the current min (or max) in O(1) and lets you swap it out in O(log n). For "k best", keep a heap of size *k*: anything worse than the heap's worst element can be discarded immediately, so you never sort the whole input.

WHEN TO USE IT

"k largest/smallest/most frequent"; merge k sorted lists; schedule by priority; repeatedly take the current extreme. **Two heaps** (a max-heap of the lower half + min-heap of the upper half) give the running *median* in O(log n) per insert.

```
import heapq
# k largest: keep a MIN-heap of size k (smallest of the k sits on top)
h = []
for x in nums:
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)   # evict the smallest
return h            # the k largest
```

COMPLEXITY

O(n log k) for Top-K (vs O(n log n) to fully sort). Building a heap from n items is O(n); each push/pop is O(log size). Quickselect finds the kth element in O(n) average if you don't need them ordered.

PITFALL

Python's `heapq` is a min-heap; for a max-heap push negated values (or wrap with a key). For "k largest" use a *min*-heap of size k (and vice-versa) — getting this backwards is the classic slip.

14 · Tree BFS & DFS `tree`

INTUITION

BFS (a queue) explores level by level — reach the shallowest first. **DFS** (recursion or a stack) plunges down one path before backtracking — natural for "compute something about each subtree." Most tree problems are "what does each node need from its children?", which is postorder DFS.

WHEN TO USE IT

BFS: level-order output, minimum depth, "by level". DFS: path sums, height/diameter, validate BST (inorder yields sorted), serialize, lowest common ancestor, any subtree aggregate.

```
# BFS level order
from collections import deque
q, levels = deque([root]), []
while q:
    level = []
    for _ in range(len(q)):       # snapshot this level's size
        node = q.popleft(); level.append(node.val)
        if node.left:  q.append(node.left)
        if node.right: q.append(node.right)
    levels.append(level)

# DFS (postorder): height of tree
def height(node):
    if not node: return 0
    return 1 + max(height(node.left), height(node.right))
```

COMPLEXITY

O(n) time (each node once). Space: BFS O(width); DFS O(height) for the call stack — O(n) worst case on a skewed tree.

PITFALL

Capture the level size *before* the inner loop in BFS. For BST validation, pass down (min, max) bounds — checking only parent > child is insufficient.

15 · Graph traversal (BFS / DFS) graph

INTUITION

A graph is nodes plus an adjacency list. Traversal is BFS/DFS plus one new thing trees don't need: a **visited** set, because graphs have cycles and multiple paths to a node. BFS from a source gives shortest paths in an *unweighted* graph (fewest edges).

WHEN TO USE IT

Connected components / islands (flood fill); reachability; shortest path in unweighted graphs/grids; cycle detection; bipartite check (2-coloring); maze problems.

```
# Count connected components via DFS
seen = set()
def dfs(u):
    seen.add(u)
    for v in adj[u]:
        if v not in seen: dfs(v)
count = 0
for u in range(n):
    if u not in seen:
        dfs(u); count += 1
```

COMPLEXITY

O(V + E) — you touch each vertex and each edge once.

PITFALL

Mark nodes visited at the right moment (on enqueue for BFS, to avoid adding duplicates). On a grid, the "graph" is implicit — neighbors are the 4 (or 8) adjacent cells; bound-check before stepping.

16 · Topological Sort graph (DAG)

INTUITION

Order tasks so every dependency comes before the things that need it. Kahn's algorithm: repeatedly take a node with *no remaining prerequisites* (indegree 0), remove it, and decrement its neighbors. If you can't order everyone, there's a cycle.

WHEN TO USE IT

Course schedules / prerequisites; build systems; task ordering with dependencies; detecting whether a dependency graph has a cycle.

```
from collections import deque
indeg = [0]*n
for u in range(n):
    for v in adj[u]: indeg[v] += 1
q = deque([u for u in range(n) if indeg[u] == 0])
order = []
while q:
    u = q.popleft(); order.append(u)
    for v in adj[u]:
        indeg[v] -= 1
        if indeg[v] == 0: q.append(v)
# len(order) < n  ⇒  a cycle exists (no valid ordering)
```

COMPLEXITY

O(V + E).

PITFALL

Topological order only exists for a DAG — always check `len(order) == n` to detect cycles. The ordering is generally not unique.

17 · Union-Find (DSU) graph

INTUITION

Maintain disjoint groups under two operations: *find* the representative of an element's group, and *union* two groups. With path compression and union by rank, both run in near-constant amortized time — perfect for "are these connected?" asked many times as edges arrive.

WHEN TO USE IT

Number of connected components; detect a cycle in an undirected graph (union fails when both ends already share a root); Kruskal's MST; dynamic connectivity; grouping/merging accounts or equivalences.

```
parent = list(range(n)); rank = [0]*n
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]   # path compression
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False           # already connected → cycle
    if rank[ra] < rank[rb]: ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]: rank[ra] += 1
    return True
```

COMPLEXITY

Effectively O(α(n)) ≈ O(1) per operation with both optimizations.

PITFALL

Without path compression *and* union by rank you can degrade toward O(n) per find. Union-Find handles *undirected* connectivity and merges; it doesn't natively support deletions or directed reachability.

18 · Dijkstra & shortest paths weighted graph

INTUITION

Greedy BFS weighted by distance: always expand the closest *unsettled* node next, pulled from a min-heap. Once a node is popped, its shortest distance is final. It's BFS where a priority queue replaces the plain queue so cost, not edge count, drives the order.

WHEN TO USE IT

Shortest/cheapest path with **non-negative** weights; network latency; cheapest flights within k stops (with tweaks). For unweighted graphs, plain BFS suffices; for negative weights use Bellman-Ford; all-pairs small graphs, Floyd-Warshall.

```
import heapq
dist = [INF]*n; dist[src] = 0
pq = [(0, src)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]: continue          # stale entry
    for v, w in adj[u]:
        if d + w < dist[v]:
            dist[v] = d + w
            heapq.heappush(pq, (dist[v], v))
```

COMPLEXITY

O(E log V) with a binary heap.

PITFALL

Dijkstra breaks with negative edges. Skip stale heap entries (the `d > dist[u]` check) instead of a decrease-key. Don't mark settled before popping.

19 · Backtracking recursion

INTUITION

Explore a decision tree depth-first: *choose* an option, *explore* the consequences recursively, then *un-choose* to try the next. Prune branches that can't lead to a valid answer as early as possible. You're systematically generating possibilities while abandoning dead ends.

WHEN TO USE IT

"Generate all" subsets/permutations/combinations; constraint puzzles (N-Queens, Sudoku, word search); partitioning; anything where you build a solution incrementally and must satisfy constraints.

```
# All subsets
res = []
def backtrack(start, path):
    res.append(path[:])               # every node is a valid subset
    for i in range(start, len(nums)):
        path.append(nums[i])          # choose
        backtrack(i + 1, path)        # explore
        path.pop()                    # un-choose
backtrack(0, [])
```

COMPLEXITY

Usually exponential — $O(2^n)$ subsets, $O(n!)$ permutations — because the answer set itself is that large. Pruning cuts the constant/branches, not the worst-case class.

PITFALL

Append a *copy* of `path`, not the live list. Use a `start` index for combinations (order doesn't matter) and a `used[]` mask for permutations. Sort first and skip equal siblings to dedupe when inputs repeat.

20 · Divide & Conquer recursion

INTUITION

Split the problem into independent halves, solve each recursively, then *combine*. The combine step is where the real work (and the cleverness) lives. Many O(n log n) algorithms are this shape: log n levels of recursion, O(n) combine work per level.

WHEN TO USE IT

Sorting (merge/quick sort); count inversions; closest pair of points; majority element; "merge results of two halves"; problems naturally expressed on subranges.

```
def merge_sort(a):
    if len(a) <= 1: return a
    mid = len(a) // 2
    left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    return merge(left, right)         # the combine step does the work
```

COMPLEXITY

Often O(n log n) (e.g. merge sort: $T(n)=2T(n/2)+O(n)$). The Master Theorem formalizes the recurrence-to-bound mapping.

PITFALL

The split must yield independent subproblems; if halves interact, you likely need DP instead. Watch recursion depth and the cost of the combine step — a careless $O(n^2)$ combine kills the benefit.

21 · Dynamic Programming — deep dive optimization

DP is the pattern people find hardest, so it gets the most space. It is just *recursion with overlapping subproblems whose answers you cache.*

INTUITION

DP applies when a problem has two properties: **overlapping subproblems** (the naive recursion recomputes the same inputs repeatedly) and **optimal substructure** (an optimal answer is built from optimal answers to subproblems). Cache each subproblem's answer once and the exponential collapses to polynomial.

HOW TO DERIVE A DP — THE RELIABLE RECIPE

1. **Define the state.** What minimal set of variables fully describes a subproblem? (e.g. `dp[i]` = best answer using the first `i` items; `dp[i][w]` = best with first `i` items and capacity `w`.) Getting the state right is 90% of DP.
2. **Write the recurrence.** Express `dp[state]` in terms of smaller states — usually "the best over the choices available here."
3. **Set base cases.** The smallest states you can answer directly.
4. **Choose an order.** Top-down memoization (recursion + cache) or bottom-up tabulation (fill an array so dependencies are ready first).

```
# Top-down (memoized) — reads like the recurrence
from functools import lru_cache
@lru_cache(None)
def dp(i, cap):
    if i == n or cap == 0: return 0
    best = dp(i+1, cap)                      # skip item i
    if weight[i] <= cap:                      # take item i
        best = max(best, value[i] + dp(i+1, cap - weight[i]))
    return best

# Bottom-up (tabulated) 0/1 knapsack
dp = [0]*(W+1)
for i in range(n):
    for c in range(W, weight[i]-1, -1):       # reverse → each item once
        dp[c] = max(dp[c], value[i] + dp[c-weight[i]])
```

THE DP SUB-PATTERNS WORTH RECOGNIZING

Sub-pattern	State shape	Examples
Linear / "house robber"	`dp[i]` from `dp[i-1], dp[i-2]`	Max non-adjacent sum, climbing stairs, decode ways
0/1 Knapsack	`dp[i][cap]`	Subset sum, partition equal subset, target sum
Unbounded knapsack	`dp[amount]`	Coin change (min coins / # ways)
2D strings	`dp[i][j]` over two sequences	Edit distance, LCS, regex/wildcard match
Subsequence	`dp[i]` = best ending at i	Longest increasing subsequence (O(n log n) w/ patience)
Grid	`dp[r][c]` from up/left	Unique paths, min path sum
Interval	`dp[i][j]` over a range	Matrix-chain, burst balloons, palindrome partition

WHEN TO USE IT

"Maximum / minimum / longest / number of ways"; sequential choices where each affects the future; a brute-force recursion that recomputes the same arguments; "can you reach / partition / make change". If greedy seems plausible but you can find a counterexample, it's usually DP.

COMPLEXITY

(number of distinct states) × (work per state). 1D often O(n); 2D often O(n·m). Space can frequently drop a dimension (keep only the last row/few values).

PITFALL

The usual failures: a state that doesn't capture everything the future depends on; a wrong iteration order (a dependency not yet computed); missing or wrong base cases; and iterating the knapsack capacity the wrong direction (forward reuses an item, reverse uses it once). When stuck, write the slow recursion first, then add memoization.

22 · Greedy optimization

INTUITION

Make the choice that looks best *right now* and never reconsider. It works only when the problem has the **greedy-choice property** — a locally optimal pick is part of some globally optimal solution. When that holds, greedy is simpler and faster than DP; when it doesn't, it's confidently wrong.

WHEN TO USE IT

Interval scheduling (pick earliest finish time); activity selection; Huffman coding; jump game; assigning with a sort + one pass. Often a sort precedes the greedy sweep.

```
# Max non-overlapping intervals: always keep the earliest finisher
intervals.sort(key=lambda x: x[1])
end, count = float('-inf'), 0
for s, e in intervals:
    if s >= end:            # doesn't overlap the last kept one
        count += 1; end = e
```

COMPLEXITY

Usually O(n log n) for the sort plus an O(n) sweep.

PITFALL

Greedy's danger is silent incorrectness. Before trusting it, justify the greedy-choice property or test a counterexample. If a local optimum can block a better global one (e.g. coin systems that aren't canonical), switch to DP.

23 · Trie strings

INTUITION

A tree where each edge is a character, so words sharing a prefix share a path. Lookups and prefix queries cost O(L) — the length of the word — independent of how many words are stored. It trades memory for fast prefix operations.

WHEN TO USE IT

Autocomplete / prefix search; dictionary word lookup; "does any word start with…"; word-search boards; replacing many words by prefix; (a binary trie) maximizing XOR pairs.

```
class Trie:
    def __init__(self): self.root = {}
    def insert(self, w):
        node = self.root
        for ch in w: node = node.setdefault(ch, {})
        node['$'] = True            # end-of-word marker
    def starts_with(self, p):
        node = self.root
        for ch in p:
            if ch not in node: return False
            node = node[ch]
        return True
```

COMPLEXITY

O(L) per insert/search/prefix query; space up to O(total characters across all words).

PITFALL

Use an explicit end-of-word marker so "app" isn't reported just because "apple" exists. For large alphabets a dict per node beats a fixed-size array on memory.

24 · Bit Manipulation numbers

INTUITION

Integers are bit-vectors you can operate on in parallel. XOR cancels duplicates (`x ^ x = 0`); a bitmask compactly represents a subset of up to ~20–30 items; shifts and masks set/clear/test individual bits in O(1).

WHEN TO USE IT

"Single number" among pairs (XOR everything); enumerate subsets via bitmask; track visited states compactly (bitmask DP); parity/counting set bits; swap or flag without extra space.

```
x ^ y            # differing bits
x & (x - 1)      # clears the lowest set bit (count bits by looping this)
x & -x           # isolates the lowest set bit
x | (1 << i)     # set bit i      x & ~(1 << i)  # clear bit i
(x >> i) & 1     # test bit i
# subset of {0..n-1} ↔ integer mask; iterate masks 0 .. (1<<n)-1
```

COMPLEXITY

O(1) per bit op; subset enumeration is O(2^n) by nature; bitmask DP is O($2^n \cdot n$) or similar.

PITFALL

Mind operator precedence (parenthesize shifts/masks) and language-specific signed-shift and overflow behavior. Bitmask tricks only scale to ~20–30 elements before 2^n explodes.

25 · The pattern cheat-sheet

One-screen recall. If you can reproduce this table from memory and explain each intuition, you've mastered the map.

Pattern	Strongest trigger	Typical cost
Two Pointers	Sorted array, pair/triplet, in-place	O(n)
Sliding Window	Contiguous subarray/substring + constraint	O(n)
Fast & Slow	Cycle / middle of linked list	O(n), O(1) space
Binary Search	Sorted, or "smallest value that works"	O(log n) / O(n log range)
Prefix Sums	Range sums, "subarray sums to k"	O(n)
Merge Intervals	Overlapping ranges	O(n log n)
Cyclic Sort	Numbers 1..n, missing/duplicate	O(n), O(1) space
Monotonic Stack	Next greater/smaller, histogram	O(n)
Heap / Top-K	k largest/smallest/frequent	O(n log k)
Two Heaps	Running median	O(log n)/insert
Tree BFS	Level-by-level	O(n)
Tree DFS	Path / subtree aggregates	O(n)
Graph BFS/DFS	Components, reachability, flood fill	O(V+E)
Topological Sort	Dependencies / ordering (DAG)	O(V+E)
Union-Find	Connectivity, undirected cycle, MST	$\sim O(\alpha) \approx O(1)$
Dijkstra	Weighted shortest path (≥0)	O(E log V)
Backtracking	Generate all / constraints	$O(2^n)$ / O(n!)
Divide & Conquer	Split-solve-combine	O(n log n) often
Dynamic Programming	Max/min/count ways, overlapping subproblems	states × work
Greedy	Local optimum is globally safe	O(n log n)
Trie	Prefix / dictionary	O(L)/op
Bit Manipulation	Single number, subset masks	O(1)/op

26 · A path to mastery

Patterns stick through deliberate practice, not passive reading. A schedule that works:

1. **Lock in foundations first.** Big-O, arrays, hash maps, recursion, and the two core traversals (BFS/DFS). Everything else builds on these.
2. **Learn one pattern at a time.** Read the intuition here, then solve 4–6 problems that use *only* that pattern back-to-back until the template is automatic. Breadth-first across patterns beats grinding random problems.
3. **Name before you code.** For every new problem, force yourself to state the pattern and the reason out loud first. This trains the recognition reflex that §4 is built around.
4. **Re-derive, don't memorize.** For DP especially, practice writing the slow recursion, then adding memoization — so you can rebuild any solution from the recipe instead of recalling code.
5. **Mix and review.** Once individual patterns are solid, do mixed sets so you practice the hardest step — *identifying* which pattern applies under uncertainty. Revisit failures after a week.
6. **Master the combinations.** Hard problems layer patterns: binary search whose check is a greedy sweep; DFS plus memoization (= DP on a graph/tree); sliding window plus a monotonic deque. Once single patterns are reflexive, study how they compose.

> **THE ONE IDEA TO KEEP**
>
> Every pattern is a way to **avoid redundant work**: two pointers and sliding windows avoid recomputing overlapping ranges; hashing avoids re-searching; prefix sums avoid re-adding; memoization avoids re-solving subproblems; heaps avoid re-sorting; Union-Find avoids re-traversing. When a brute force feels wasteful, ask "what work am I repeating, and which structure remembers it?" That question is the whole game.
